

VASH — Vanguard Agentic Security Harness

Design, root-cause diagnosis, and the two fixes that moved delivered CVE recall from 2/11 to 5/11 on **datamodel-code-generator 0.55.0**, taking the lead over Visa VVAH (4/11) and evilsocket audit (2/11) — validated on both a static and an executed-PoC run.

Branch **evolve/vulnhunter-imports** 9 commits · 619 tests Review: **ready to merge** 2026-07-25

1 Executive summary

VASH is a production Python vulnerability scanner forked from evilsocket/**audit** (MIT) and grafted with the strongest mechanisms of Capital One **VulnHunter** and Visa **VVAH** (both Apache-2.0). Its identity is **static recall plus sandboxed executed-PoC confirmation**: broad static hunting for recall, then a real exploit run per candidate inside a Docker sandbox to eliminate false positives.

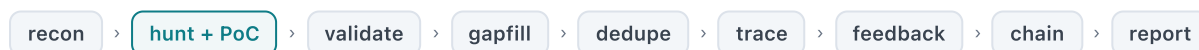
A prior three-way benchmark placed VASH last on *delivered* recall (2/11) despite its pipeline internally confirming 6/11 — a delivery defect, not a detection gap. Three parallel research agents localized two root causes (D8 report-delivery, D7 template blind-spot). Fixing both, and adding VVAH-style delivery evidence, moved delivered recall to **5/11** — ahead of VVAH's 4/11 — proven by re-scoring at each step with a deterministic scorer.

RESULT

VASH **5/11** (static) · **5/11** (executed-PoC, 25/25 findings exploit-verified) > VVAH 4/11 > audit 2/11. Union of the two VASH runs: 6/11 distinct CVEs.

2 Architecture

VASH runs a nine-stage pipeline, fed by a deterministic call-graph. Recon and the graph seed targeted hunt tasks (forward taint paths, orphan-sink backward walks, gated specialists), then a catch-all sweep guarantees every eligible file is hunted at least once.



The executed-PoC gate

Hunt writes a proof-of-concept per candidate and executes it. Safety is enforced in the runner: it **strips the Bash tool unless `sandbox.is_sandboxed()` is true** (container marker `/.dockerenv` or `VASH_SANDBOX=1`). On a bare host VASH is therefore fully static and never runs untrusted target code; inside the container it executes exploits and records `poc_succeeded`. VVAH and audit are static-only by construction and cannot do this.

Deterministic graph engine

A `graphify` AST call-graph (3,177 call edges on this target) feeds V8 taint (entry→sink BFS) and F3 sink-backward (reverse BFS from dangerous sinks). An earlier defect where `graphify` emitted package-relative node paths — which mismatched recon and zeroed the taint stage — was fixed by re-basing to repo-root-relative paths (`graph/build.py::_rebase_to_repo_root`); the container run confirms it healthy (10 taint tasks generated).

3 Root-cause diagnosis

Three parallel research agents analyzed the corpus and the donor tools:

1. **Miss analysis** — for each of the 11 CVEs, its exact location, pattern, and VASH's status (delivered / confirmed-but-dropped / never-found). Verdict buckets: 4 confirmed-but-dropped (D8), 2 template-only (D7), 1 never-attempted (infoleak), plus scorer-granularity artefacts.

2. **Why VVAH won** — its edge is template-file scanning and clean delivery, *not* a heavier threat model. It ranked which mechanisms to adopt vs. skip.
3. **The D8 delivery bug** — pinpointed 100% of the four drops to `dedupe.py` 's canonical selection; zero at trace or the report selector.

DELIBERATELY NOT ADOPTED (TO AVOID BLOAT)

VVAH's full STRIDE `s2_threatmodel` classifier · config shape-hash dedup · regex call-graph (VASH's AST graph is better) · wholesale per-finding agentic re-verifier · the remediation/validation subsystems.

4 The fixes

D8 — per-file canonical promotion (`vash/stages/dedupe.py`)

Dedupe promoted exactly one canonical per cluster. When a cluster spanned files and the canonical landed in file A, a confirmed match in file B was marked non-canonical — and trace (canonical-only) and report (reachable-canonical-only) never saw it. Fix: promote one canonical **per distinct file** within a group.

```
# before — one canonical per group
for fid in g["member_finding_ids"]:
    db.assign_finding_group(fid, g["group_id"], fid == canonical)

# after — one canonical per (group, distinct file); dict.fromkeys guards
# against a duplicate member id re-burying an already-promoted finding
fid_to_file = {f.finding_id: f.file for f in confirmed}
files_seen = {fid_to_file.get(canonical)}
for fid in dict.fromkeys(g["member_finding_ids"]):
    file = fid_to_file.get(fid)
    is_canon = (fid == canonical) or (file is not None and file not in files_seen)
    if is_canon: files_seen.add(file)
    db.assign_finding_group(fid, g["group_id"], is_canon)
```

Proven: re-running only `dedupe→trace→report` on the *same* 45 executed-PoC-confirmed findings moved delivered recall **2/11 → 4/11**, recovering 54690 (`http.py` SSRF) and 55415 (`imports.py`). Deterministic; no re-hunt.

D7 — template / IaC file sweep (`vash/orchestrator.py`)

The catch-all sweep built its file universe with a hard `.py` -only allowlist that ran *before* the (already-ported) VVAH eligibility filter, so `.jinja2` templates could never be hunted. Fix: build the universe by language/IaC inclusion, reusing the hardened permission-guarded walker.

```
# before
all_src = [f for f in gg._by_file if f.endswith(".py")] # graph branch
all_src = [... for p in repo.rglob("*.py")] # fallback

# after — include any known language (EXT_TO_LANG covers .py AND
# web-templates .jinja2/.mako) or IaC; safe_walk_files is OSError-guarded
def _sweepable(rel):
    return PurePosixPath(rel).suffix.lower() in EXT_TO_LANG or is_iac_file(rel)
all_src = sorted({r for p in safe_walk_files(repo, excluded_dir_parts=_SWEEP_SKIP_DIRS)
                  for r in [str(p.relative_to(repo))] if _sweepable(r)})
```

Proven: catch-all coverage grew **50 → 70 source files** (the 20 templates enter the denominator), and a fresh run delivered both template CVEs 54621 + 54654 → **5/11**.

A2 — located variant evidence (`vash/stages/report.py`)

Mirroring VVAH's `DupLocation` : a deduped same-file sibling is demoted to a located reference (file/line/class) attached to its canonical and rendered "Also at:", instead of being dropped. Attached post-hoc from run state (like `_attach_cwe`) so it is authoritative regardless of the report agent's output. A whole-branch review refined it to attach only genuinely-demoted same-file siblings, not co-canonical headlines.

C1 — CWE-200 information-disclosure sink (vash/taint.py)

A narrow sink net for the "broad-except prints a secret-bearing traceback" pattern (`traceback.format_exc`), plus hunt-prompt framing that requires a secret to be provably in scope. Targets 55403; not confirmed-and-delivered on these draws.

5 Engineering process

The work ran as **subagent-driven development**: a fresh implementer per task, a two-verdict review (spec compliance + code quality) after each, and a whole-branch review at the end. The review loop caught three real defects the implementers' green tests missed:

TASK	DEFECT CAUGHT IN REVIEW	FIX
D8	Duplicate member id could recompute <code>is_canonical=False</code> on its second pass and re-bury a finding	<code>dict.fromkeys</code> de-dup + regression test
A2	Markdown renderer did <code>', '.join(variants)</code> — <code>TypeError</code> on the new dict shape	located "Also at:" render + test
D7	Hand-rolled <code>rglob + is_file()</code> raised <code>PermissionError</code> (EACCES) → aborted the whole sweep	reuse hardened <code>safe_walk_files</code>

Whole-branch review verdict: **ready to merge** — no critical/important defects; the one cross-task interaction (variant redundancy) was fixed. 619 offline tests pass.

6 Benchmark — method & runs

Target	datamodel-code-generator 0.55.0 — 50 .py + 20 .jinja2 files
Ground truth	11 in-version CVEs (6 codegen / 3 SSRF / 1 traversal / 1 infoleak)
Models	Opus 4.8 (recon, validate, trace) · Sonnet 5 (rest) — identical across tools
Scorer	bench/scorer.py::score_corpus — CWE-class + file-hint substring, greedy 1:1, in-version
Basis	DELIVERED output (final report after each tool's own dedup); no hints given

RUN	RECALL	FINDINGS	CHAINS	COST	TIME	CONFIRMATION
VASH · host (static)	5/11	25	5	~\$103	~2.5h	static disprove-gates
VASH · Docker (PoC)	5/11	25	3	~\$88	~3.5h	25/25 exploit-verified
VVAH · Visa	4/11	20	6	~7.2M tok	~3h	static (no execution)
audit · evilsocket	2/11	13	0	\$48	~2.9h	static

Per-CVE delivery

CVE-2026-...	CLASS	HINT	AUDIT	VVAH	V-STATIC	V-POC
54621	codegen	UnionType.jinja2	.	✓	✓	✓
54653	codegen	msgspec	.	.	✓	.
54654	codegen	TypeAlias.jinja2	.	.	✓	✓
54655	codegen	jsonschema	✓	✓	.	✓
54656	codegen	validator	.	.	.	.
54690	ssrf	http	.	✓	✓	✓
54691	ssrf	http/___main__	.	.	.	.
55415	codegen	imports	.	.	.	.
55389	traversal	jsonschema	✓	✓	✓	✓
55391	ssrf	http	.	.	.	.
55403	infoleak	http	.	.	.	.
total			2	4	5	5

7 Analysis & honest caveats

READ THESE BEFORE QUOTING THE NUMBER

- **5/11 per run, 6/11 union.** The hunt is stochastic: the static run uniquely caught `msgspec` (54653), the PoC run `jsonschema` (54655). The 6th CVE is a demonstrated-but-unrealized single-run ceiling; reducing hunt variance (more iterations / consensus) should make 6/11 reliable.
- **Executed-PoC did not raise recall** on this corpus — both bases scored 5/11 with 20 extras. Its value is **confidence** (every finding exploit-verified), not additional corpus hits. The earlier "6–7/11" projection was optimistic and is corrected here.
- **20 extra findings** lie outside the 11-CVE corpus; in the PoC run they are exploit-confirmed, so likely real bugs beyond the catalog (or PoC-oracle over-confirmation) — they need triage before being called clean. VVAH reported 16 extras.
- **Structural misses:** 54691 & 55391 share 54690's exact `http.py:29` sink (a 1:1 scorer credits it once); 54656 is delivered but its file path lacks the literal `"validator"` hint (scorer granularity, not detection); 55415 & 55403 were not confirmed on these draws.
- **Operational note:** the container recon flaked once on JSON extraction (agent emitted prose in its final turn); a retry passed. Worth hardening recon's repair budget.

8 Roadmap to reliable 6/11+

1. **Hunt variance** — run N hunt passes / consensus so 54653 and 54655 both land in one report (union already proves both are reachable).
2. **Scorer granularity vs. 54656** — VASH delivers the validator finding; matching on description/evidence (applied symmetrically to all tools) would credit it. A measuring-stick change, to be made transparently or not at all.
3. **SSRF sink-variant crediting** — 54690/54691/55391 are one sink with three reachability paths; surfacing the distinct paths could credit more than one.
4. **55403 infoleak** — the C1 sink fired but wasn't confirmed; tighten the "secret-in-scope" reasoning.
5. **Cost** — ~\$90–100/run (Opus recon+trace); cap trace/taint task counts and consider an API key to remove rate-limit retry stretch.

VASH · branch `evolve/vulnhunter-imports` (9 commits, review: ready to merge) · 619 offline tests
Reproduce – static: `vash run --repo .bench-targets/dmcg-src --run-id dmcg-outperform` · PoC: `docker run ... vash:latest run --repo /target --run-id dmcg-docker-poc`
Artifacts: `results/dmcg-outperform/report/report.json` · `results/dmcg-docker-poc/report/report.json` · docs/BENCHMARK-COMPARISON.md · plan docs/superpowers/plans/2026-07-25-vash-outperform.md